We optionally show `next_x`.

```
if next_x is not None:
    ax[1].scatter(
        next_x[..., 0],
        next_x[..., 1],
        c="r",
        marker="*",
        s=500,
        label="next query"
    )
```

4. Copy the GP class from Chapter 3's exercise, which implements a Matérn 2.5 kernel with ARD. Further modify this class to make it integratable with BoTorch.

```
class GPModel(gpytorch.models.ExactGP, botorch.models.gpytorch.GPyTorchModel): ❶
    _num_outputs = 1 ❶

    def __init__(self, train_x, train_y, likelihood):
        super().__init__(train_x, train_y, likelihood)
        self.mean_module = gpytorch.means.ConstantMean()
        self.covar_module = gpytorch.kernels.ScaleKernel( ❷
            gpytorch.kernels.MaternKernel( ❷
                nu=2.5, ❷
                ard_num_dims=2 ❷
            ) ❷
        ) ❷

    def forward(self, x):
        ... ❸
```

❶ BoTorch-related modifications.

❷ Matérn 2.5 kernel with ARD.

❸ Omitted.

5. Reuse helper function `fit_gp_model()` and the for loop that implements BayesOpt. We copy `fit_gp_model()` and declare the initial data set.

```
train_x = torch.tensor([
    [1., 1.],
])
train_y = f(train_x)
```

We then declare the BayesOpt loop:

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>